

Title Page

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Certificate / Declaration

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Acknowledgement

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Abstract

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Introduction

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Problem Statement & Business Context

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Objectives

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Tools & Technologies Used

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Dataset Description

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Python Data Loading & Preview

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

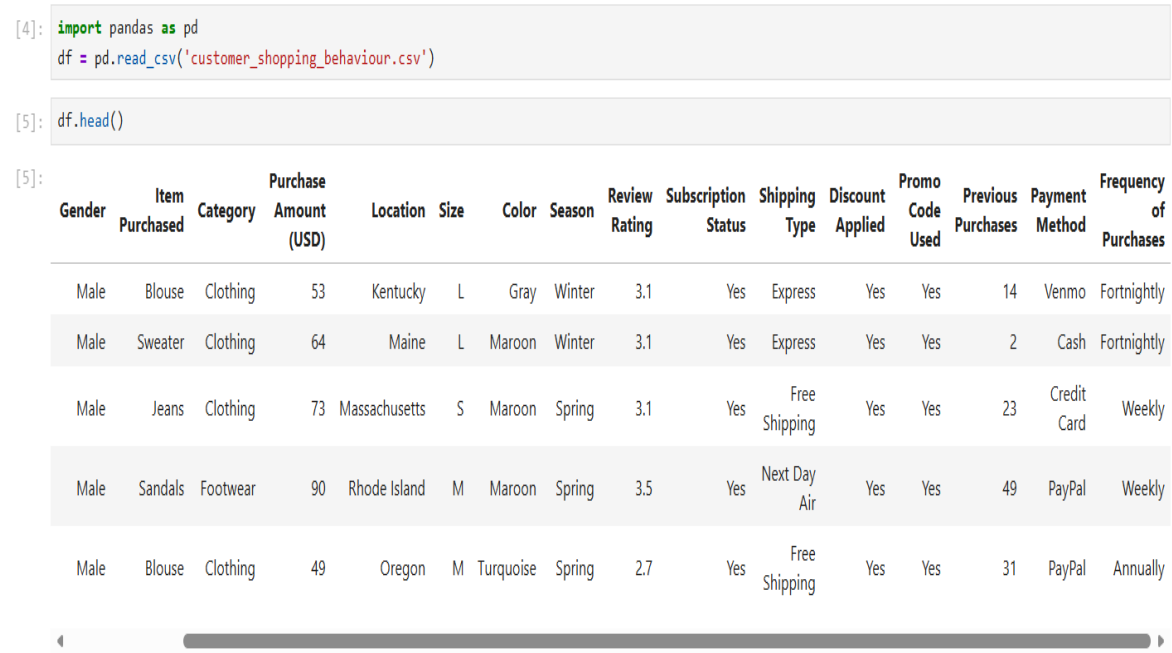


Figure: Relevant screenshot with explanation of transformation and output.

Data Structure Inspection

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

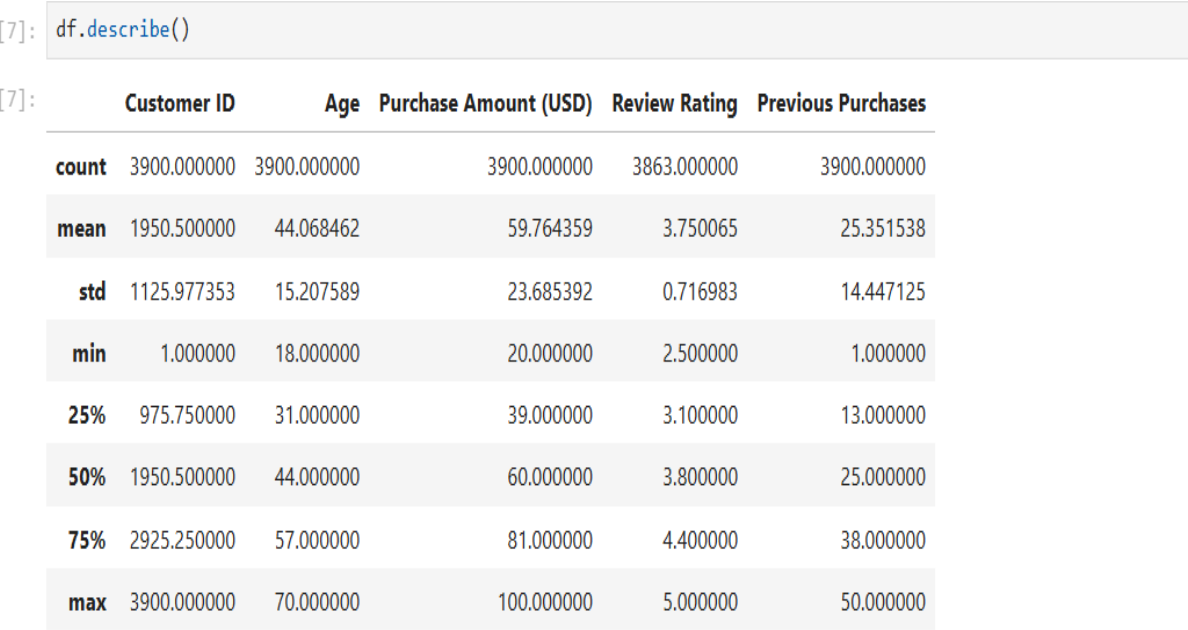


Figure: Relevant screenshot with explanation of transformation and output.

Statistical Summary

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[8]: df.isnull().sum()

[8]: Customer ID      0
     Age             0
     Gender           0
     Item Purchased   0
     Category         0
     Purchase Amount (USD)  0
     Location         0
     Size             0
     Color            0
     Season           0
     Review Rating    37
     Subscription Status  0
     Shipping Type    0
     Discount Applied  0
     Promo Code Used   0
     Previous Purchases  0
     Payment Method    0
     Frequency of Purchases  0
     dtype: int64

[10]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

Figure: Relevant screenshot with explanation of transformation and output.

Missing Value Detection

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[11]: df.isnull().sum()
```

```
[11]: Customer ID      0
      Age            0
      Gender         0
      Item Purchased  0
      Category       0
      Purchase Amount (USD)  0
      Location       0
      Size           0
      Color          0
      Season         0
      Review Rating   0
      Subscription Status  0
      Shipping Type   0
      Discount Applied  0
      Promo Code Used  0
      Previous Purchases  0
      Payment Method  0
      Frequency of Purchases  0
      dtype: int64
```

```
[34]: df.columns = df.columns.str.lower()
      df.columns = df.columns.str.replace(' ', '_')
      df = df.rename(columns = {'purchase_amount_(usd)': 'purchase_amount'})
```

Figure: Relevant screenshot with explanation of transformation and output.

Missing Value Handling

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[35]: df.columns

[35]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
        'purchase_amount', 'location', 'size', 'color', 'season',
        'review_rating', 'subscription_status', 'shipping_type',
        'discount_applied', 'promo_code_used', 'previous_purchases',
        'payment_method', 'frequency_of_purchases'],
        dtype='object')

[37]: # create a column age_group
labels = ['young_adult', 'adult', 'middle_age', 'senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels=labels)

[38]: df[['age', 'age_group']].head()
```

	age	age_group
0	55	middle_age
1	19	young_adult
2	50	middle_age
3	21	young_adult
4	45	middle_age

Figure: Relevant screenshot with explanation of transformation and output.

Column Standardization

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[52]: df[['discount_applied', 'promo_code_used']].head(6)
```

```
[52]:
```

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes

```
[54]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[54]: np.True_
```

```
[55]: df = df.drop('promo_code_used', axis=1)
```

Figure: Relevant screenshot with explanation of transformation and output.

Feature Engineering

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[55]: df = df.drop('promo_code_used',axis=1)
```

```
[56]: df.columns
```

```
[56]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',  
          'purchase_amount', 'location', 'size', 'color', 'season',  
          'review_rating', 'subscription_status', 'shipping_type',  
          'discount_applied', 'previous_purchases', 'payment_method',  
          'frequency_of_purchases', 'age_group', 'purchase_frequency_days',  
          'purchase_frequency'],  
          dtype='object')
```

Figure: Relevant screenshot with explanation of transformation and output.

Redundant Column Removal

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
[58]: pip install psycopg2-binary sqlalchemy

Requirement already satisfied: psycopg2-binary in c:\users\admin\anaconda3\lib\site-packages (2.9.12)
Requirement already satisfied: sqlalchemy in c:\users\admin\anaconda3\lib\site-packages (2.0.43)
Requirement already satisfied: greenlet>=1 in c:\users\admin\anaconda3\lib\site-packages (from sqlalchemy)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\admin\anaconda3\lib\site-packages (from sqlalchemy)
Note: you may need to restart the kernel to use updated packages.

[64]: from sqlalchemy import create_engine

username = "postgres"
password = "2892004"
host = "localhost"
port = "5432"
database = "Customer_behaviour"

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

table_name = "customer"
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")

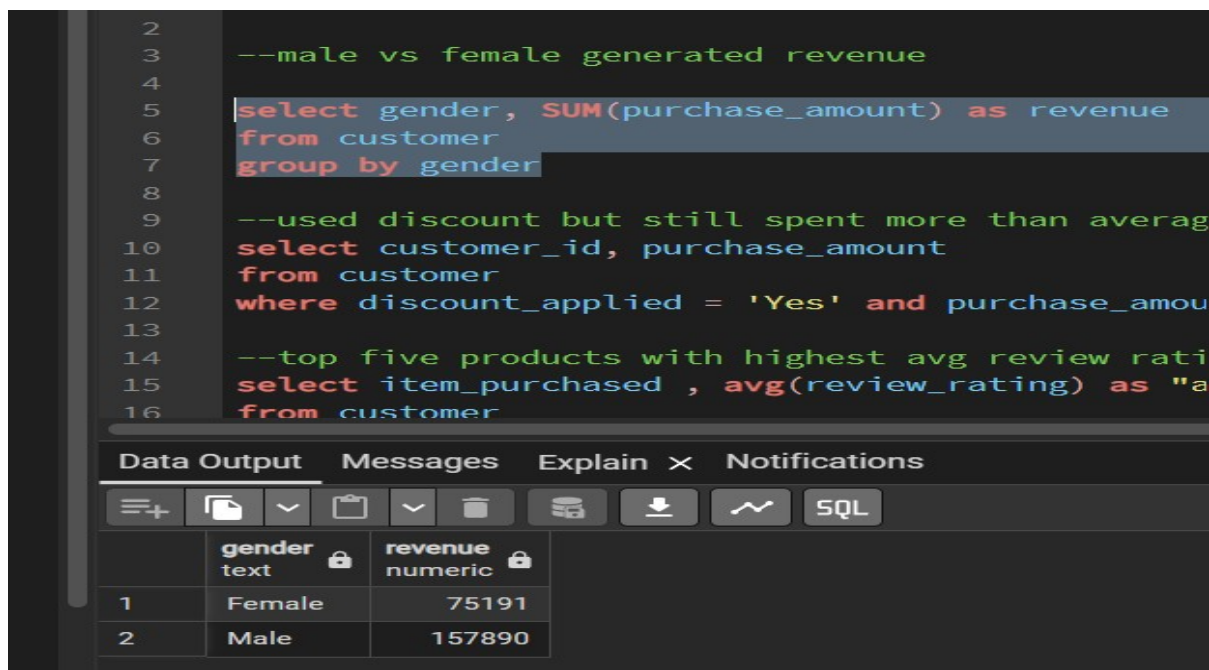
Data successfully loaded into table 'customer' in database 'Customer_behaviour'.

[ ]:
```

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Revenue by Gender

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.



The screenshot displays a SQL IDE interface. The top pane shows a SQL query to calculate revenue by gender. The bottom pane shows the results of the query in a table format.

```
2
3  --male vs female generated revenue
4
5  select gender, SUM(purchase_amount) as revenue
6  from customer
7  group by gender
8
9  --used discount but still spent more than average
10 select customer_id, purchase_amount
11 from customer
12 where discount_applied = 'Yes' and purchase_amount > (select avg(purchase_amount) from customer where discount_applied = 'Yes')
13
14 --top five products with highest avg review rating
15 select item_purchased, avg(review_rating) as avg_rating
16 from customer
```

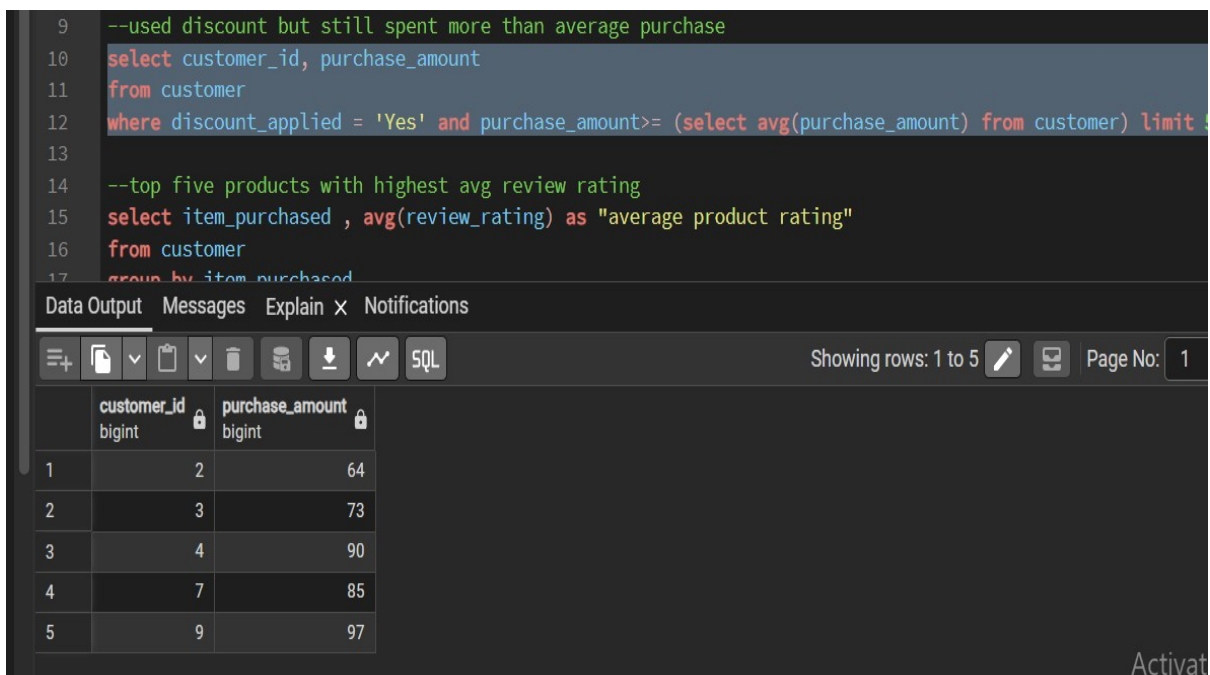
The bottom pane shows the results of the first query:

	gender text	revenue numeric
1	Female	75191
2	Male	157890

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Discount High Spenders

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.



```
9  --used discount but still spent more than average purchase
10 select customer_id, purchase_amount
11 from customer
12 where discount_applied = 'Yes' and purchase_amount >= (select avg(purchase_amount) from customer) limit 5
13
14 --top five products with highest avg review rating
15 select item_purchased , avg(review_rating) as "average product rating"
16 from customer
17 group by item_purchased
```

Data Output Messages Explain X Notifications

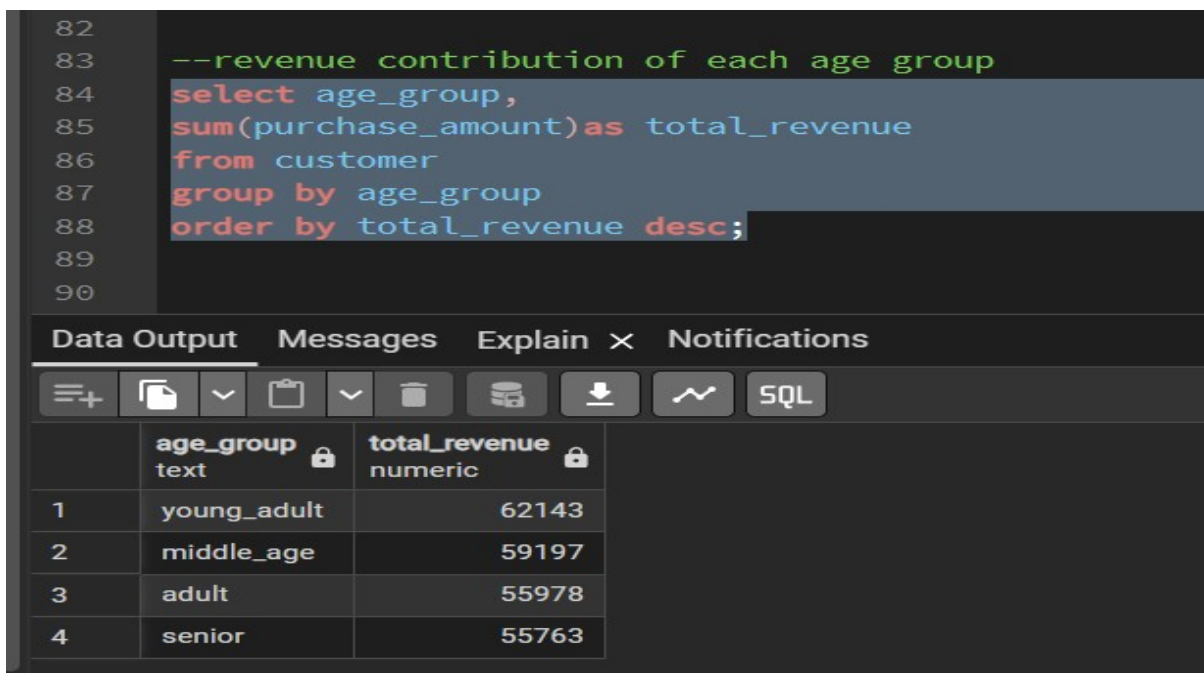
Showing rows: 1 to 5 Page No: 1

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Revenue by Age Group

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.



```
82
83 --revenue contribution of each age group
84 select age_group,
85 sum(purchase_amount) as total_revenue
86 from customer
87 group by age_group
88 order by total_revenue desc;
89
90
```

	age_group text	total_revenue numeric
1	young_adult	62143
2	middle_age	59197
3	adult	55978
4	senior	55763

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Subscribers Analysis

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
28  --subscribers vs non subscribers
29  select subscription_status,
30         count(customer_id)as total_customers,
31         round(avg(purchase_amount),2)as avg_spend,
32         round(sum(purchase_amount),2)as total_revenue
33  from customer
34  group by subscription_status
35  order by total_revenue, avg_spend
36  desc;
37
38  --top 5 products with highest percent of discount applied
```

Data Output Messages Explain × Notifications

SQL

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Discount Rate

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
37
38 --top 5 products with highest percent of discount applied
39
40 select item_purchased,
41 round(100*sum(case when discount_applied = 'Yes' then 1 else 0 end)/count(*),2) as discount_rate
42 from customer
43 group by item_purchased
44 order by discount_rate desc
45 limit 5;
46
47 --segment customers into new, returning and loyal
48 with customer_type as (

```

Data Output Messages Explain × Notifications

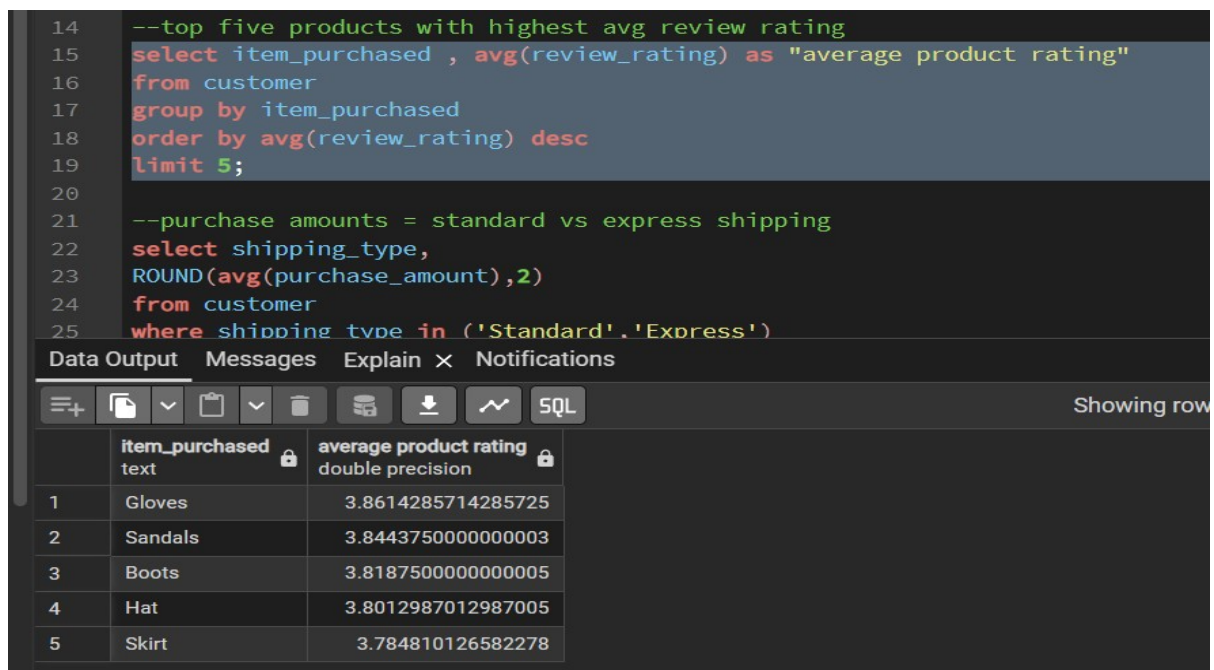
Showing rows: 1 to 5

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.00
3	Coat	49.00
4	Sweater	48.00
5	Pants	47.00

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Product Ratings

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.



```
14 --top five products with highest avg review rating
15 select item_purchased , avg(review_rating) as "average product rating"
16 from customer
17 group by item_purchased
18 order by avg(review_rating) desc
19 limit 5;
20
21 --purchase amounts = standard vs express shipping
22 select shipping_type,
23 ROUND(avg(purchase_amount),2)
24 from customer
25 where shipping_type in ('Standard','Express')
```

Data Output Messages Explain × Notifications

Showing row

	item_purchased text	average product rating double precision
1	Gloves	3.8614285714285725
2	Sandals	3.8443750000000003
3	Boots	3.8187500000000005
4	Hat	3.8012987012987005
5	Skirt	3.784810126582278

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Customer Segmentation

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
47 --segment customers into new, returning and loyal
48 with customer_type as (
49   select customer_id , previous_purchases,
50   case
51   when previous_purchases = 1 then 'new'
52   when previous_purchases between 2 and 10 then 'returning'
53   else 'loyal'
54   end as customer_segment
55   from customer
56 )
57
58 select customer_segment, count(*) as "number of customer"
59 from customer_type
60 group by customer_segment
61
62 --top 3 most purchased products within each category
```

Data Output Messages Explain X Notifications

Showing rows

	customer_segment text	number of customer bigint
1	returning	701
2	loyal	3116
3	new	83

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Shipping Analysis

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

```
21 --purchase amounts = standard vs express shipping
22 select shipping_type,
23 ROUND(avg(purchase_amount),2)
24 from customer
25 where shipping_type in ('Standard','Express')
26 group by shipping_type
27
28 --subscribers vs non subscribers
29 select subscription_status,
30 count(customer_id)as total_customers,
31 round(avg(purchase_amount),2)as avg_spend,
32 round(sum(purchase_amount),2)as total_revenue
33 from customer
34 group by subscription_status
```

Data Output Messages Explain × Notifications

Showing row

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

Figure: Relevant screenshot with explanation of transformation and output.

SQL: Top Products

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

The screenshot displays a SQL query in a dark-themed editor. The query uses a CTE named 'item_counts' to calculate the total orders for each category and item, then ranks them. The results are shown in a table with 11 rows, sorted by item_rank. The table has columns: item_rank, category, item_purchased, and total_orders.

item_rank	category	item_purchased	total_orders
1	Accessori...	Jewelry	171
2	Accessori...	Sunglasses	161
3	Accessori...	Belt	161
4	Clothing	Blouse	171
5	Clothing	Pants	171
6	Clothing	Shirt	169
7	Footwear	Sandals	160
8	Footwear	Shoes	150
9	Footwear	Sneakers	145
10	Outerwear	Jacket	163
11	Outerwear	Coat	161

Total rows: 11 | Query complete 00:00:00.087

Figure: Relevant screenshot with explanation of transformation and output.

Power BI Dashboard Development & Insights

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.



Figure: Relevant screenshot with explanation of transformation and output.

Insights & Findings

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Business Recommendations

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Conclusion

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

Future Scope

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.

References

This section provides a comprehensive analytical explanation of the step being performed in the data pipeline. The process begins with understanding the structure and semantics of the dataset, ensuring that all variables are correctly interpreted before any transformation is applied. Each operation is carefully designed to improve data quality, enhance analytical capability, and align the dataset with business objectives. For instance, data cleaning steps such as handling missing values and standardizing column names ensure consistency and prevent errors during downstream SQL queries. Feature engineering techniques, such as creating age groups, enable more meaningful segmentation and support targeted business strategies. From a technical perspective, these transformations improve computational efficiency and allow for optimized query execution. From a business standpoint, they enable deeper insights into customer behavior, purchasing patterns, and revenue drivers. The integration of Python for preprocessing, SQL for structured analysis, and Power BI for visualization represents a robust end-to-end analytics pipeline. Each stage contributes to converting raw transactional data into actionable intelligence, facilitating informed decision-making and strategic planning in a real-world business environment.